

SQL JOINS

Project: Doctor Appointment Booking System

Database: appointment_db

Aim: To demonstrate different types of SQL joins using the appointment database.

1. Source Tables Used for JOIN Operations

The JOIN queries use the patients, doctors and appointments tables. The available screenshots of the source tables are included below.

Patients Table

The screenshot shows the MySQL Workbench interface with the 'appointment_db' schema selected. The 'patients' table is highlighted in the Navigator. The SQL editor shows the following SQL code:

```
VALUES
(1, 'Deeraj', 'deeraj@gmail.com', '9876543210', 19, 'Male'),
(2, 'Rahul', 'rahul@gmail.com', '9876543211', 20, 'Male'),
(3, 'Ananya', 'ananya@gmail.com', '9876543212', 21, 'Female');
SELECT * FROM patients;
```

The Result Grid displays the following data:

patient_id	patient_name	email	phone	age	gender
1	Deeraj	deeraj@gmail.com	9876543210	19	Male
2	Rahul	rahul@gmail.com	9876543211	20	Male
3	Ananya	ananya@gmail.com	9876543212	21	Female

The Output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
12	09:14:30	CREATE TABLE patients (patient_id INT PRIMARY KEY, patient_name VAR...	Error Code: 1050. Table 'patients' already exists	0.016 sec
13	09:16:36	CREATE TABLE doctors (doctor_id INT PRIMARY KEY, doctor_name VARC...	0 row(s) affected	0.125 sec
14	09:18:46	ALTER TABLE patients ADD gender VARCHAR(10)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.422 sec
15	09:19:21	DESCRIBE patients	6 row(s) returned	0.016 sec / 0.000 sec
16	09:20:27	INSERT INTO patients (patient_id, patient_name, email, phone, age, gender) VALU...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.047 sec
17	09:20:49	SELECT * FROM patients LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

Doctors Table

The screenshot shows the MySQL Workbench interface with the 'appointment_db' schema selected. The 'doctors' table is highlighted in the Navigator. The SQL editor shows the following SQL code:

```
VALUES
(101, 'Dr. Raj Kumar', 'Cardiology', 800.00),
(102, 'Dr. Priya Sharma', 'Dermatology', 600.00),
(103, 'Dr. Arjun Rao', 'Neurology', 1000.00);
SELECT * FROM doctors;
```

The Result Grid displays the following data:

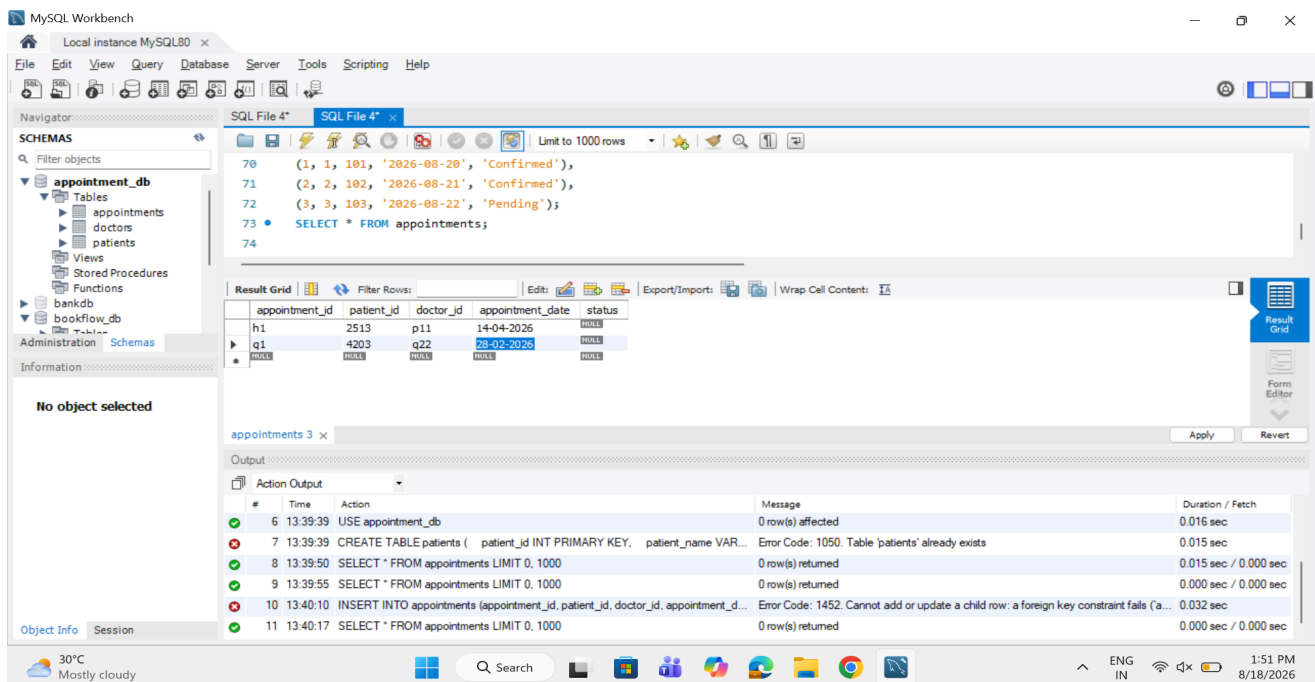
doctor_id	doctor_name	specialization	consultation_fee
101	Dr. Raj Kumar	Cardiology	800.00
102	Dr. Priya Sharma	Dermatology	600.00
103	Dr. Arjun Rao	Neurology	1000.00

The Output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
14	09:18:46	ALTER TABLE patients ADD gender VARCHAR(10)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.422 sec
15	09:19:21	DESCRIBE patients	6 row(s) returned	0.016 sec / 0.000 sec
16	09:20:27	INSERT INTO patients (patient_id, patient_name, email, phone, age, gender) VALU...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.047 sec
17	09:20:49	SELECT * FROM patients LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
18	09:21:25	INSERT INTO doctors (doctor_id, doctor_name, specialization, consultation_fee) V...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.031 sec
19	09:21:33	SELECT * FROM doctors LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

2. Appointments Table

The appointments table connects patients and doctors through patient_id and doctor_id. The provided screenshot is included as the practical evidence.



3. INNER JOIN

Returns only records with matching values in the joined tables.

```
SELECT p.patient_name, d.doctor_name, d.specialization,
a.appointment_date, a.status
FROM appointments a
INNER JOIN patients p ON a.patient_id = p.patient_id
INNER JOIN doctors d ON a.doctor_id = d.doctor_id;
```

4. LEFT JOIN

Returns all records from the left table and matching records from the right table.

```
SELECT p.patient_name, a.appointment_date, a.status
FROM patients p
LEFT JOIN appointments a ON p.patient_id = a.patient_id;
```

5. RIGHT JOIN

Returns all records from the right table and matching records from the left table.

```
SELECT d.doctor_name, d.specialization, a.appointment_date, a.status
FROM appointments a
RIGHT JOIN doctors d ON a.doctor_id = d.doctor_id;
```

6. FULL OUTER JOIN

MySQL does not directly provide FULL OUTER JOIN, so LEFT JOIN and RIGHT JOIN can be combined with UNION.

```
SELECT p.patient_name, a.appointment_id, a.status
FROM patients p
LEFT JOIN appointments a ON p.patient_id = a.patient_id
UNION
SELECT p.patient_name, a.appointment_id, a.status
FROM patients p
RIGHT JOIN appointments a ON p.patient_id = a.patient_id;
```

7. CROSS JOIN

Returns every possible combination of rows from the two tables.

```
SELECT p.patient_name, d.doctor_name
FROM patients p
CROSS JOIN doctors d;
```

8. SELF JOIN

A SELF JOIN joins a table with itself. It can be used to find doctors having the same specialization.

```
SELECT d1.doctor_name AS doctor_1,
d2.doctor_name AS doctor_2,
d1.specialization
FROM doctors d1
JOIN doctors d2
ON d1.specialization = d2.specialization
AND d1.doctor_id < d2.doctor_id;
```

Conclusion

Thus, INNER, LEFT, RIGHT, FULL OUTER, CROSS and SELF JOIN operations were documented using the Doctor Appointment Booking System tables.

Note: The screenshots included in this document are the available source-table evidence. Join-result screenshots can be added later after executing each JOIN query.